

UNIT34: GLTF MODEL - RENDER MESHES

【学習要項】

- ☐ Render meshes
- ☐ SoA vs AoS

【演習手順】

1. シェーダーの実装

① HLSL ヘッダーファイルの追加 (glTF_model.hlsl)

```
1: struct VS_IN
2: {
3:     float4 position : POSITION;
4:     float4 normal : NORMAL;
5:     float4 tangent : TANGENT;
6:     float2 texcoord : TEXCOORD;
7:     uint4 joints : JOINTS;
8:     float4 weights : WEIGHTS;
9: };
10:
11: struct VS_OUT
12: {
13:     float4 position : SV_POSITION;
14:     float4 w_position : POSITION;
15:     float4 w_normal : NORMAL;
16:     float4 w_tangent : TANGENT;
17:     float2 texcoord : TEXCOORD;
18: };
19:
20: cbuffer PRIMITIVE_CONSTANT_BUFFER : register(b0)
21: {
22:     row_major float4x4 world;
23:     int material;
24:     bool has_tangent;
25:     int skin;
26:     int pad;
27: };
28:
29: cbuffer SCENE_CONSTANT_BUFFER : register(b1)
30: {
31:     row_major float4x4 view_projection;
32:     float4 light_direction;
33:     float4 camera_position;
34: };
```

② 頂点シェーダーファイルの追加 (glTF_model_vs.hlsl)

```
1: #include "glTF_model.hlsl"
2:
3: VS_OUT main(VS_IN vin)
4: {
5:     VS_OUT vout;
6:
7:     vin.position.w = 1;
8:     vout.position = mul(vin.position, mul(world, view_projection));
9:     vout.w_position = mul(vin.position, world);
10:
11:     vin.normal.w = 0;
12:     vout.w_normal = normalize(mul(vin.normal, world));
13:
14:     float sigma = vin.tangent.w;
15:     vin.tangent.w = 0;
16:     vout.w_tangent = normalize(mul(vin.tangent, world));
17:     vout.w_tangent.w = sigma;
18:
19:     vout.texcoord = vin.texcoord;
20: }
```

UNIT34: GLTF MODEL - RENDER MESHES

```
21: return vout;
22: }
```

③ ピクセルシェーダーファイルの追加 (gltf_model_ps.hlsl)

```
1: #include "gltf_model.hlsl"
2:
3: float4 main(VS_OUT pin) : SV_TARGET
4: {
5:     float3 N = normalize(pin.w_normal.xyz);
6:     float3 L = normalize(-light_direction.xyz);
7:
8:     float3 color = max(0, dot(N, L));
9:     return float4(color, 1);
10: }
```

2. シェーダーオブジェクトの定義

① gltf_model クラスのメンバ変数を定義する

```
1: Microsoft::WRL::ComPtr<ID3D11VertexShader> vertex_shader;
2: Microsoft::WRL::ComPtr<ID3D11PixelShader> pixel_shader;
3: Microsoft::WRL::ComPtr<ID3D11InputLayout> input_layout;
4: struct primitive_constants
5: {
6:     DirectX::XMFLOAT4X4 world;
7:     int material{ -1 };
8:     int has_tangent{ 0 };
9:     int skin{ -1 };
10:    int pad;
11: };
12: Microsoft::WRL::ComPtr<ID3D11Buffer> primitive_cbuffer;
```

② gltf_model クラスのコンストラクタでシェーダーオブジェクトの生成を行う

※頂点バッファは SoA (Structure of Array) で構成される (従来は AoS (Array of Structures))

```
1: D3D11_INPUT_ELEMENT_DESC input_element_desc[]
2: {
3:     { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0, D3D11_INPUT_PER_VERTEX_DATA, 0 },
4:     { "NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 1, 0, D3D11_INPUT_PER_VERTEX_DATA, 0 },
5:     { "TANGENT", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 2, 0, D3D11_INPUT_PER_VERTEX_DATA, 0 },
6:     { "TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 3, 0, D3D11_INPUT_PER_VERTEX_DATA, 0 },
7:     { "JOINTS", 0, DXGI_FORMAT_R16G16B16A16_UINT, 4, 0, D3D11_INPUT_PER_VERTEX_DATA, 0 },
8:     { "WEIGHTS", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 5, 0, D3D11_INPUT_PER_VERTEX_DATA, 0 },
9: };
10: create_vs_from_cso(device, "gltf_model_vs.cso", vertex_shader.ReleaseAndGetAddressOf(),
11:     input_layout.ReleaseAndGetAddressOf(), input_element_desc, _countof(input_element_desc));
12: create_ps_from_cso(device, "gltf_model_ps.cso", pixel_shader.ReleaseAndGetAddressOf());
13:
14: D3D11_BUFFER_DESC buffer_desc{};
15: buffer_desc.ByteWidth = sizeof(primitive_constants);
16: buffer_desc.Usage = D3D11_USAGE_DEFAULT;
17: buffer_desc.BindFlags = D3D11_BIND_CONSTANT_BUFFER;
18: HRESULT hr;
19: hr = device->CreateBuffer(&buffer_desc, nullptr, primitive_cbuffer.ReleaseAndGetAddressOf());
20: _ASSERT_EXPR(SUCCEEDED(hr), hr_trace(hr));
```

3. gltf_model クラスに render メンバ関数を実装する

```
1: void gltf_model::render(ID3D11DeviceContext* immediate_context, const DirectX::XMFLOAT4X4& world)
2: {
3:     using namespace DirectX;
4:
5:     immediate_context->VSSetShader(vertex_shader.Get(), nullptr, 0);
6:     immediate_context->PSSetShader(pixel_shader.Get(), nullptr, 0);
7:     immediate_context->IASetInputLayout(input_layout.Get());
```

UNIT34: GLTF MODEL - RENDER MESHES

```
8:   immediate_context->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
9:
10:  std::function<void(int)> traverse{ [&](int node_index)->void {
11:      const node& node{nodes.at(node_index)};
12:      if (node.mesh > -1)
13:      {
14:          const mesh& mesh{ meshes.at(node.mesh) };
15:          for (std::vector<mesh::primitive>::const_reference primitive : mesh.primitives)
16:          {
17:              ID3D11Buffer* vertex_buffers[]{
18:                  primitive.has("POSITION") ?
19:                      buffers.at(primitive.vertex_buffer_views.at("POSITION").buffer).Get() : NULL,
20:                  primitive.has("NORMAL") ?
21:                      buffers.at(primitive.vertex_buffer_views.at("NORMAL").buffer).Get() : NULL,
22:                  primitive.has("TANGENT") ?
23:                      buffers.at(primitive.vertex_buffer_views.at("TANGENT").buffer).Get() : NULL,
24:                  primitive.has("TEXCOORD_0") ?
25:                      buffers.at(primitive.vertex_buffer_views.at("TEXCOORD_0").buffer).Get() : NULL,
26:                  primitive.has("JOINTS_0") ?
27:                      buffers.at(primitive.vertex_buffer_views.at("JOINTS_0").buffer).Get() : NULL,
28:                  primitive.has("WEIGHTS_0") ?
29:                      buffers.at(primitive.vertex_buffer_views.at("WEIGHTS_0").buffer).Get() : NULL,
30:              };
31:              UINT strides[]{
32:                  primitive.has("POSITION") ?
33:                      static_cast<UINT>(primitive.vertex_buffer_views.at("POSITION").stride_in_bytes) : 0,
34:                  primitive.has("NORMAL") ?
35:                      static_cast<UINT>(primitive.vertex_buffer_views.at("NORMAL").stride_in_bytes) : 0,
36:                  primitive.has("TANGENT") ?
37:                      static_cast<UINT>(primitive.vertex_buffer_views.at("TANGENT").stride_in_bytes) : 0,
38:                  primitive.has("TEXCOORD_0") ?
39:                      static_cast<UINT>(primitive.vertex_buffer_views.at("TEXCOORD_0").stride_in_bytes) : 0,
40:                  primitive.has("JOINTS_0") ?
41:                      static_cast<UINT>(primitive.vertex_buffer_views.at("JOINTS_0").stride_in_bytes) : 0,
42:                  primitive.has("WEIGHTS_0") ?
43:                      static_cast<UINT>(primitive.vertex_buffer_views.at("WEIGHTS_0").stride_in_bytes) : 0,
44:              };
45:              UINT offsets[]{
46:                  primitive.has("POSITION") ?
47:                      static_cast<UINT>(primitive.vertex_buffer_views.at("POSITION").byte_offset) : 0,
48:                  primitive.has("NORMAL") ?
49:                      static_cast<UINT>(primitive.vertex_buffer_views.at("NORMAL").byte_offset) : 0,
50:                  primitive.has("TANGENT") ?
51:                      static_cast<UINT>(primitive.vertex_buffer_views.at("TANGENT").byte_offset) : 0,
52:                  primitive.has("TEXCOORD_0") ?
53:                      static_cast<UINT>(primitive.vertex_buffer_views.at("TEXCOORD_0").byte_offset) : 0,
54:                  primitive.has("JOINTS_0") ?
55:                      static_cast<UINT>(primitive.vertex_buffer_views.at("JOINTS_0").byte_offset) : 0,
56:                  primitive.has("WEIGHTS_0") ?
57:                      static_cast<UINT>(primitive.vertex_buffer_views.at("WEIGHTS_0").byte_offset) : 0,
58:              };
59:              immediate_context->IASetVertexBuffers(0, _countof(vertex_buffers), vertex_buffers, strides, offsets);
60:
61:              primitive_constants primitive_data{};
62:              primitive_data.material = primitive.material;
63:              primitive_data.has_tangent = primitive.has("TANGENT");
64:              primitive_data.skin = node.skin;
65:              XMStoreFloat4x4(&primitive_data.world,
66:                  XMLoadFloat4x4(&node.global_transform) * XMLoadFloat4x4(&world));
67:              immediate_context->UpdateSubresource(primitive_cbuffer.Get(), 0, 0, &primitive_data, 0, 0);
68:              immediate_context->VSSetConstantBuffers(0, 1, primitive_cbuffer.GetAddressOf());
69:              immediate_context->PSSetConstantBuffers(0, 1, primitive_cbuffer.GetAddressOf());
70:
71:              if (primitive.index_buffer_view.buffer > -1)
72:              {
```

UNIT34: GLTF MODEL - RENDER MESHES

```
73:         immediate_context->IASetIndexBuffer(buffers.at(primitive.index_buffer_view.buffer).Get(),
74:         primitive.index_buffer_view.format, static_cast<UINT>(primitive.index_buffer_view.byte_offset));
75:         immediate_context->DrawIndexed(static_cast<UINT>(primitive.index_buffer_view.count), 0, 0);
76:     }
77:     else
78:     {
79:         immediate_context->Draw(static_cast<UINT>(primitive.vertex_buffer_views.at("POSITION").count), 0);
80:     }
81: }
82: }
83: for (std::vector<int>::value_type child_index : node.children)
84: {
85:     traverse(child_index);
86: }
87: } };
```

```
88: for (std::vector<int>::value_type node_index : scenes.at(default_scene).nodes)
89: {
90:     traverse(node_index);
91: }
92: }
```

4. gltf_model クラスの render メンバ関数を呼び出し、モデルが描画されることを確認する

※glTF のシーンは基本的に右手系・Y 軸アップで記録されている

※別のモデルデータでもテストする

【評価項目】

☐モデルの描画